

Параметры конфигурации сервисного модуля

При реализации сервисного модуля может потребоваться необходимость в перестройке логики его функциональной части в зависимости от окружения, в котором он развёрнут, от изменения требований, предъявляемых пользователем, от необходимости дополнительной диагностики его работы и прочих факторов.

Этой цели служат параметры конфигурации приложения, задаваемые в ini-файлах приложения и/или сервисов, а также у облачных сервисов в "Управлении облаком" в реестре "[Настройки](#)". Для поддержания централизованного управления настройкой приложения и возможности подстройки логики модуля без непосредственного участия его автора(-ов) **важно** описывать все используемые модулем параметры в его метаданных с помощью [Genie](#).



Значения параметров можно использовать из прикладного кода с помощью синглтона [sbis::Config](#) / [sbis.Config](#). Подробнее об этом читайте в разделе [Использование параметров из прикладного кода](#).

Структура конфигурации приложения

Источники конфигурации

Конфигурация приложения времени выполнения наполняется из различных источников, основными из которых являются (в порядке увеличения приоритета, более приоритетные источники переопределяют значения менее приоритетных):

1. Метаописание приложения

Имена параметров конфигурации, типы их значений и значения по умолчанию загружаются из метаописаний сервисных модулей, входящих в состав приложения:

Genie Project Edit Report Settings distributed-locks SDK: 25.6000 4 Deploy

distributed-locks > distributed-locks > Middleware > Distributed Locks Service

Distributed Lo...

Module customization

^ Main

Name Distributed Locks Service

Comment Сервис распределённых блокировок.

Package Middleware

Kaizen and responsible Select area of responsibility Бадаев Е.Н.

Component Select component

Library sbis-distributed-locks-service

Is Delay Load of Module Libraries

Os Windows Linux iOS MacOS Android

Link type Dynamic Static

^ Ports

^ Module dependencies

Module name	Os	OS Flavour	Compiler	Architecture	Link type	Configuration
Redis Client	Windows, Linux, iOS, M...	Centos7, RedHat9	Clang, clang15, Apple-...	i386, i686, x86-64, Pow...	Dynamic, Static	Release, Address Saniti...

^ Load current module after following modules

^ Config parameters

Search

Name	Use existing parameter	Type	Comment
Middleware services.Distributed Locks.CollisionGuardTimeout	false	Number	Длительность защиты идентификаторов захвата от колли...
Middleware services.Distributed Locks.ListKeysResultLimit	false	Number	Устанавливает предельное количество извлекаемых ключ...
Middleware services.Distributed Locks.LockReasonLimit	false	Number	Максимально допустимая длина поля "Причина установки...
Middleware services.Distributed Locks.LockTimeout	false	Number	Таймаут истечения неподтверждаемого захвата блокиров...
Middleware services.Distributed Locks.RequestHoldupTimeout	false	Number	Длительность удержания запроса захвата блокировки в о...
Middleware services.Distributed Locks.RevocationTimeout	false	Number	Ограничение на частоту проверки истечения захваченных...
Middleware services.Distributed Locks.ScriptBatchLimit	false	Number	Ограничение на количество команд исполнения Lua-скрип...
Middleware services.Distributed Locks.WritePreferenceTimeout	false	Number	Интервал защиты запросов на захват блокировки записи о...

^ Resources

Search Events Gulp's Logs

Переопределения значений параметров загружаются из метаописаний сервисов приложения:

Genie Project Edit Report Settings distributed-locks SDK: 25.6000 4 Deploy

distributed-locks > distributed-locks

distributed-lo...

Service customization

Settings **Extended**

^ Main

^ Parameter values

Name	Type	Values
Ядро.Сервер приложений.МаксимальноеЧислоВнутреннихКорутин	Number	all: 0

Multiplexor policy Any

И из метаописания самого приложения:

Genie Project Edit Report Settings distributed-locks SDK: 25.6000 4 Deploy

distributed-locks

distributed-lo...

Cloud properties

Name	distributed-locks
Kind	None
Application family	Middleware Bus
SDK version	25.6000
Comment	Сервис распределённых блокировок.
Kaizen and responsible	Select area of responsibility Бадаев Е.Н. x

DB Templates +

Search paths +

Parameter values +

Name	Type	Values
Ядро.Сервер приложений.ОграничениеНаЧислоКорутин	Number	all: 1000
Ядро.Сервер приложений.ПределИспользованияПамяти	Number	all: 1280
Ядро.Сервер приложений.РазмерСтекаКорутины	Number	all: 256
Ядро.Сервер приложений.ТипОсновногоПула	String	all: embedded
Ядро.Сервер приложений.ТипПланировщикаКорутин	String	all: template
Ядро.Сервер приложений.ЧислоПотоковВРабочихПроцессах	Number	all: 4

Additional CMake file

2. Облачная конфигурация

Облачные сервисы в качестве основного источника используют конфигурацию, поставляемую сервисом [Управления конфигурациями](#) в составе Оркестратора облака. Настольные и мобильные приложения не используют облачную конфигурацию в штатной работе.

О том, как изменять конфигурацию облачного приложения через реестр [Настройки](#), можно узнать в [Руководстве пользователя](#).

3. Конфигурационные ini-файлы

В дополнение к переопределениям значений параметров в дистрибутиве приложения, рассмотренному выше, значения параметров могут быть заданы в ini-файлах, как входящих в состав дистрибутива, так и расположенных по известным путям:

- 1 **[Базовая конфигурация]**
- 2 ПаузаПередЗагрузкойМодулей = 30
- 3 **[Ядро.Права]**
- 4 Проверять = Нет
- 5 **[Ядро.Сервер приложений]**
- 6 ИмяСлужбы = sbis-cfg

В ini-файлах имя параметра разделяется на 2 сегмента: подстрока до последней точки образует имя секции параметров и указывается в квадратных скобках, остальная часть — имя ключа в секции. А секция **Базовая конфигурация** зарегистрирована для параметров, не содержащих точек в имени (такие параметры создавать в прикладном коде запрещено).

Тип параметра в ini-файлах не указывается и определяется автоматически при загрузке конфигурации по метаописанию приложения или значению, присвоенному параметру в ini-файле.

Объединение конфигурационных ini-файлов

В настольных и мобильных приложениях, имеющих региональные и/или брендированные версии, есть необходимость переиспользования большей части значений параметров, указанных в конфигурационных файлах, во всех вариантах исполнения, подстраивая под регион или бренд лишь небольшое количество параметров.

Платформа для этого предоставляет нестандартную функциональность разбора ini-файлов: поддержку включения файлов друг в друга директивой **#include** (аналогично таким языкам программирования, как C/C++).

- 1 #include base.ini

- 2
- 3 [Ядро.Права]
- 4 Проверять = Нет

Директивы `#include` могут располагаться в любом месте ini-файла. Ограничений на их количество и вложенность так же не установлено.

Пути включаемых конфигурационных файлов разрешаются относительно пути включающего ini-файла.

Слои конфигурации

В настольных и мобильных приложениях конфигурация содержится в нескольких ini-файлах и разделена на несколько перекрывающихся друг друга слоёв. Такая организация позволяет отделять пользовательские переопределения параметров от конфигурации приложения по умолчанию и задавать различные значения параметрам в разных обособленных сервисах (пулах) приложения.

1. Базовые настройки приложения (Application)

Слой содержит базовые значения параметров приложения. Распространяется на все процессы приложения: процесс-координатор и все рабочие процессы всех пулов приложения.

Настройки хранятся в файле `sbis-rpc-service.ini` в облачных сервисах или `sbis-config.ini` в мобильных и настольных приложениях.

2. Пользовательские настройки приложения (ApplicationCustom)

Слой содержит пользовательские переопределения базовых значений параметров приложения. Распространяется на все процессы приложения.

Настройки хранятся в файле `user-config.ini`.

3. Общие настройки приложения (Common)

Слой содержит переопределения базовых значений параметров, общие для всех пользователей приложения. Перекрывает индивидуальные пользовательские настройки. Распространяется на все процессы приложения.

Настройки хранятся в файле `common-config.ini`.

4. Настройки пула приложения (Pool)

Слой содержит переопределения значений параметров пула приложения. Распространяется только на рабочие процессы конкретного пула приложения.

Настройки хранятся в файле `<имя-пула>.ini`.

5. Пользовательские настройки пула приложения (PoolCustom)

Слой содержит пользовательские переопределения значений параметров пула приложения. Распространяется только на рабочие процессы конкретного пула приложения.

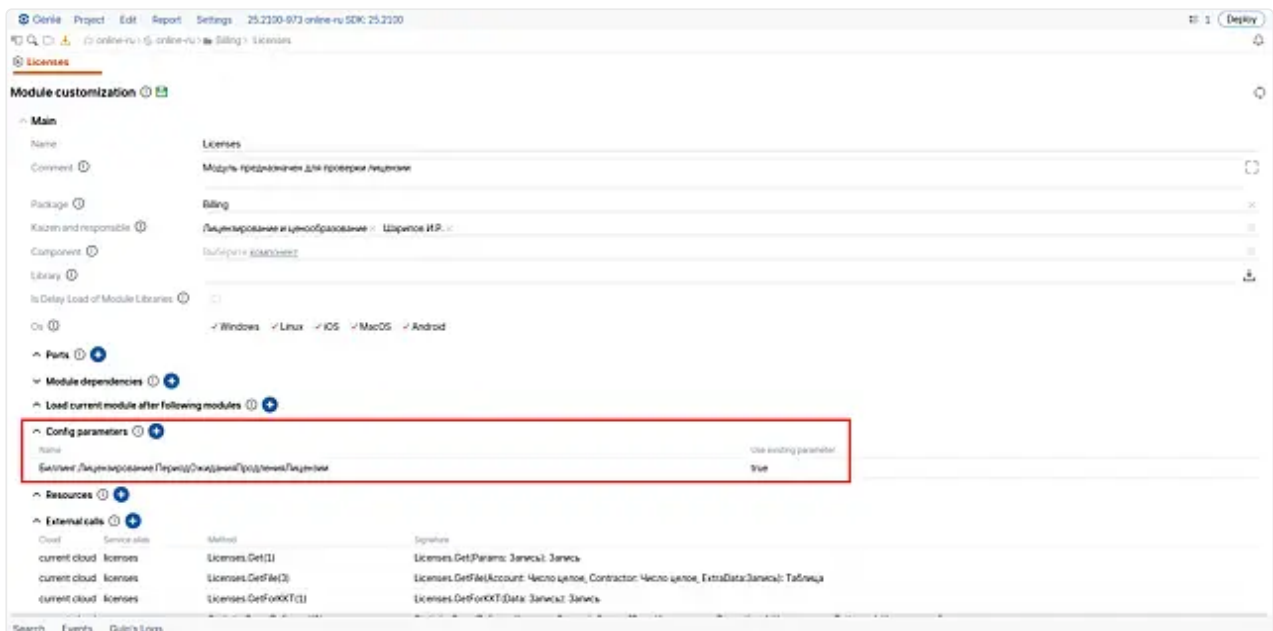
Настройки хранятся в файле `<имя-пула>-user-config.ini`.

Создание параметра конфигурации

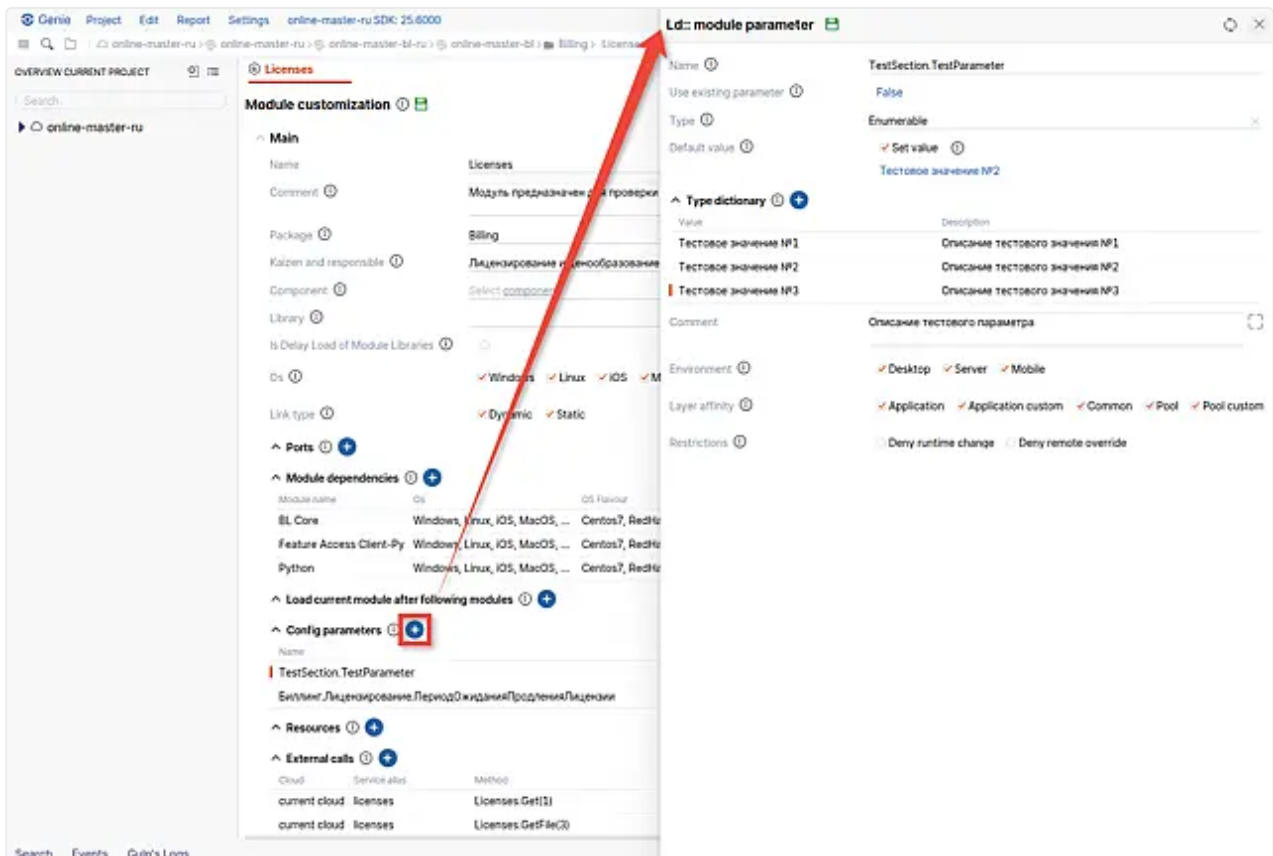


Подробнее о создании параметра описано [здесь](#).

1. Перейдите в настройки сервисного модуля (**Module customization**) и найдите раздел **Config parameters**. В нем уже могут быть созданы некоторые параметры:



2. В разделе **Config parameters** нажмите "+" для добавления нового параметра или кликните на строку параметра для редактирования существующего.



В диалоговом окне **Module parameter** заполните атрибуты:

- **Название (Name)** — название параметра, должны соблюдаться правила именования.
- **Использовать существующий параметр (Use existing parameter)** — признак того, что используется параметр, описанный в другом модуле.
- **Тип значения (Type)** — тип данных значения параметра: строка, число, логическое, массив, объект и т.п.
- **Единица измерения (Unit)** — используется для параметров, значения которых требуют понимания, в каких единицах измерения они заданы. Единицы измерения рекомендуется задавать в национальных условных обозначениях ОКЕИ.
- **Значение по умолчанию (Default value)** — значение параметра, которое будет использовано, если для сервиса/приложения не задано конкретного переопределения значения. Рекомендуется задавать всем параметрам, иначе возможна множественная дубликация значения при задании из кода с потенциальным расхождением при необходимости его изменения.
- **Словарь значений (Type dictionary)** — список допустимых значений и их описаний параметра типа *Перечисляемое (Enumerable)*.

- **Комментарий (Comment)** — содержит комментарий об использовании параметра. Обратите внимание, что комментарии являются обязательными, должны быть содержательными и отражать все аспекты работы с данным параметром.
- **Тип приложения (Environment)** — выбор типа приложения, для которого предназначен параметр: Desktop, Server или Mobile. Параметры без флага Server не будут регистрироваться в облаке в реестре "Настройки".
- **Привязка к слоям конфигурации (Layer affinity)** — указывает в каких слоях конфигурации разрешено переопределение значения параметра.
- **Ограничения (Restrictions)** — ограничения применимости, накладываемые на параметр. Такие как:
 - Запрет на изменение значения параметра во время работы приложения (Deny runtime change);
 - Запрет на удалённое переопределения значения параметра (Deny remote override).

3. Созданный параметр отобразится в модуле. Далее его значение можно задать в сервисе "Управление облаком", а также обращаться к этому значению из [кода](#).

Имя параметра

1. Параметры должны иметь название, как минимум, из трех секций, разделенных точками, например, "Ядро.БесшовноеОбновление.РаботатьВРежимеОбновления". При регистрации этого параметра в реестре "[Настройки](#)" будет создан в корне раздел "Ядро", если он не был создан ранее, в нем подраздел "БесшовноеОбновление", а в нем уже листом "РаботатьВРежимеОбновления".



2. Параметры должны находиться в каком-либо разделе. Нельзя добавлять параметры непосредственно в "корень" реестра. Сначала нужно указать корневой раздел, внутри которого можно добавить подразделы и далее добавляется сам параметр.
3. Корневые разделы параметров должны быть согласованы с ответственным за [участок "Настройки облака"](#), и включены в перечень разрешённых корневых разделов. Для такого согласования отдельного регламента нет, поэтому пишется сообщение, либо поручение ответственному за участок. Если корневой раздел отсутствует в списке разрешённых, при регистрации будет выводиться ошибка с требованием исправить.

Более подробно правила именования параметров описаны [здесь](#).

Типы данных

- **Строка (String)** — произвольная строка.
- **Число (Number)** — целое число, ограниченное 64-битным знаковым диапазоном, или действительное, ограниченное диапазоном чисел двойной точности IEEE 754.
- **Логическое (Boolean)** — значения "Да" или "Нет".
- **Пароль (Password)** — произвольная строка, используемая для аутентификации. В отличие от других строковых типов, значение типа "пароль" скрывается при логировании.
- **Таблица паролей (Password table)** — словарь с реквизитами аутентификации, ключи — строки имён пользователей, значения — строки паролей (в формате одноуровневого объекта JSON). Как и значения типа "пароль" скрывается при логировании.

- **Перечисляемое (Enumerable)** — строка с ограниченной областью допустимых значений.
- **Дата и время (Date time)** — момент времени, представленный как дата и время суток.
- **Дата (Date)** — момент времени, представленный только датой.
- **Время (Time)** — момент времени, представленный только временем суток.
- **Объект (Object)** — произвольное значение в формате JSON.
- **Массив строк (String array)** — массив произвольных строк (в формате одноуровневого массива JSON).
- **Массив чисел (Number array)** — массив целых чисел, ограниченных 64-битным знаковым диапазоном, или действительных, ограниченных диапазоном чисел двойной точности IEEE 754 (в формате одноуровневого массива JSON).
- **Точка подключения к кластеру (Cluster endpoint)** — описание точки подключения к некоторому кластеру серверов. В виде JSON представленного ниже формата (где все поля первого уровня опциональны):

```

1  [
2    {
3      "credentials": {                                // Реквизиты
аутентификации
4        "user_role_1": {                              // Имя
пользовательской роли
5          "login": "user 1",                          // Login для
аутентификации
6          "password": "password 1"                    // Пароль для
аутентификации
7        },
8        "user_role_2": {
9          "login": "user 2",
10         "password": "password 2"
11       }
12     },
13     "id": "1129866",                                // Идентификатор
мастера в Облаке
14     "login": "",                                    // Login для
аутентификации (устарело)
15     "master": "master.cluster.tensor.ru:6379",      // Строка
подключения к мастеру
16     "params": {},                                    // Параметры
подключения
17     "password": "",                                  // Пароль для
аутентификации (устарело)
18     "protocol": "",                                  // Протокол
взаимодействия
19     "replication_scheme": "master-replica",          // Применяемая
схема репликации
20     "slave_dns": "slaves.cluster.tensor.ru:6379",   // Строка
подключения к сервису реплик
21     "slaves": [                                      // Список серверов-
реплик
22       {
23         "id": "1129867",                              // Идентификатор
реплики в Облаке
24         "slave": "slave1.cluster.tensor.ru:6379"    // Строка
подключения к реплике
25       },
26       {
27         "id": "1129868",
28         "slave": "slave2.cluster.tensor.ru:6379"

```

```

29         },
30         {
31             "id": "1129869",
32             "slave": "slave3.cluster.tensor.ru:6379"
33         }
34     ],
35     "timeout": 5000,                                // Таймаут операций
36     "type": "redis"                                  // Тип кластера
37 }
38 ]

```

Использование параметров из прикладного кода

Чтобы обратиться из прикладного кода к параметрам конфигурации сервисных модулей, используются методы синглтона **Config** ([C++](#)|[Python](#)).

Получение значения параметра

Получение стабильной ссылки на текущее значение параметра или значение по умолчанию

Если параметру присвоено значение в конфигурационных ini-файлах, установлено в облачной конфигурации или задано в коде, вернётся ссылка на него. Иначе будет возвращена ссылка на значение по умолчанию, указанное в метаописании приложения.

Если значение не задано и либо параметр отсутствует в метаописании, либо у него не указано значение по умолчанию, будет выброшено исключение.

Функциональность поддерживается только в C++ и служит целям оптимизации: возвращаемая ссылка гарантированно валидна всё время жизни приложения и позволяет не копировать тяжёлые значения параметров конфигурации.

- C++

```

1 double const& value = Config::Instance().GetRef< double >(
  L"Section.Parameter"_sv );

```

Получение текущего значения параметра или значения по умолчанию

Если параметру присвоено значение в конфигурационных ini-файлах, установлено в облачной конфигурации или задано в коде, вернётся его копия. Иначе будет возвращена копия значения по умолчанию, указанного в метаописании приложения.

Если значение не задано и либо параметр отсутствует в метаописании, либо у него не указано значение по умолчанию, будет выброшено исключение.

- C++

```

1 double value = Config::Instance().Get< double >(
  L"Section.Parameter"_sv );

```

Получение текущего значения параметра или указанного значения

Если параметру присвоено значение в конфигурационных ini-файлах, установлено в облачной конфигурации или задано в коде, вернётся его копия. Иначе будет возвращена копия значения, переданного вторым аргументом.

- C++

```

1 double value = Config::Instance().Get< double >(
  L"Section.Parameter"_sv, 3.14 );

```

- Python

```

1 value: float = Config.Instance().GetFloat('Section.Parameter', 3.14)

```

Получение текущего значения параметра или ничего

Если параметру присвоено значение в конфигурационных ini-файлах, установлено в облачной конфигурации или задано в коде, вернётся его копия. Иначе будет возвращено "ничто": `boost::none` в C++ или `None` в Python.

- C++

```
1 boost::optional< double > value
2   = Config::Instance().TryGet< double >( L"Section.Parameter"_sv );
3 if( value )
4     ...
```

- Python

```
1 value: Optional[float] =
  Config.Instance().TryGetFloat('Section.Parameter')
2 if value is not None:
3     ...
```

Получение текущего значения параметра или выброс исключения

Если параметру присвоено значение в конфигурационных ini-файлах, установлено в облачной конфигурации или задано в коде, вернётся его копия. Иначе будет выброшено исключение.

- C++

```
1 double value = Config::Instance().GetOrThrow< double >(
  L"Section.Parameter"_sv );
```

- Python

```
1 value: float = Config.Instance().GetOrThrowFloat('Section.Parameter')
```

Получение строкового представления текущего значения параметра

Все методы извлечения типизированного значения параметра, описанные ранее, могут быть использованы для получения строкового представления значения. Для этого в C++ следует передавать аргумент шаблона `String`, а в Python использовать методы с суффиксом `String`:

- C++

```
1 String value = Config::Instance().Get< String >(
  L"Section.Parameter"_sv, L"3.14"_s );
```

- Python

```
1 value: str = Config.Instance().GetOrThrowString('Section.Parameter')
```

В дополнение можно с той же целью использовать метод `Get`, он вернёт установленное параметру значение или пустую строку, если значение явно не задано:

- C++

```
1 String value = Config::Instance().Get( L"Section.Parameter"_sv );
```

- Python

```
1 value: str = Config.Instance().Get('Section.Parameter')
```

Или метод `GetOrThrow`, который выбросит исключение, если значение явно не задано:

- C++

```
1 String value = Config::Instance().GetOrThrow( L"Section.Parameter"_sv
  );
```

- Python

```
1 value: str = Config.Instance().GetOrThrow('Section.Parameter')
```

Установка значения параметра

В некоторых сценариях необходимо изменить значения параметра из кода приложения:

- изменение настройки пользователем;
- значение параметра вычислимо только во время исполнения приложения;
- тестирование функциональности приложения.

Для этих целей предоставляется метод `Set` :

- `C++`

```
1 Config::Instance().Set( L"Section.Parameter"_sv, 3.14 );
2
3 // Для установки параметра строкового типа следует явно указывать
  специализацию.
4 Config::Instance().Set< String >( L"Section.AnotherParameter"_sv,
  L"foobar"_s );
```

- `Python`

```
1 Config.Instance().Set('Section.Parameter', 3.14)
```

Устанавливать значения параметру по изменению настройки пользователем следует в соответствующий слой пользовательской конфигурации:

- `C++`

```
1 Config::Instance().Set( L"Section.Parameter"_sv, 3.14,
  ConfigLayer::ApplicationCustom );
```

- `Python`

```
1 Config.Instance().Set('Section.Parameter', 3.14,
  ConfigLayer.ApplicationCustom)
```

Сохранение пользовательской конфигурации

Микросервисам, предоставляющим конечному пользователю возможность изменять логику работы приложения, после установки пользовательского значения параметру, как правило, требуется сохранить пользовательскую конфигурацию. Чтобы после перезапуска приложения изменённые пользователем настройки не сбросились на умолчания.

Для сохранения пользовательской конфигурации следует использовать функцию `SaveUserConfig` с указанием слоя конфигурации, в который вносились изменения:

- `C++`

```
1 // Сохранение пользовательских настроек приложения по стандартному
  расположению
2 // user-config.ini.
3 SaveUserConfig();
4
5 // Сохранение пользовательских настроек пула приложения.
6 SaveUserConfig( pool_user_config_filename, ConfigLayer::PoolCustom );
```

Временное переопределение значений параметров

Автономным приложениям подсистема конфигурирования предоставляет возможность временного переопределения значений параметров с помощью микросервиса `config-override`. Эта функциональность в первую очередь ориентирована на механизм удалённого (облачного) изменения конфигурации автономных приложений в целях предотвращения и/или ликвидации аварий, связанных с некорректным взаимодействием автономных приложений с облачными сервисами.

Подсистема конфигурирования рассылает полученные переопределения параметров всем рабочим процессам приложения и применяет их в каждом процессе, в том числе в процессе-координаторе, в зависимости от приоритета

переопределения:

- Переопределения с нормальным приоритетом (Normal) применяются, только если значение переопределяемого параметра не изменено пользовательской конфигурацией приложения.
- Переопределения с высоким приоритетом (High) вступают в силу незамедлительно.

Микросервис `config-override` сохраняет текущие переопределения параметров в постоянном хранилище и заново загружает и применяет их рано при повторном запуске приложения. Это позволяет изменять поведение ранних стадий загрузки приложения.

Переопределения задаются функцией `OverrideConfiguration`.

- C++

```
1 // Формируем словарь переопределений.
2 // В типичном сценарии этот словарь приходит с Облака.
3 config::override::Settings settings;
4 // Заказываем переопределение значения параметра Section.Parameter с
  нормальным
5 // приоритетом на значение "3.14".
6 settings.emplace( L"Section.Parameter"_s,
7                   config::override::Value{
8                     config::override::Priority::Normal,
9                     L"3.14"_s } );
10 // Заказываем переопределение значения параметра
   Section.AnotherParameter с высоким
11 // приоритетом на значение "foobar".
12 settings.emplace( L"Section.AnotherParameter"_s,
13                   config::override::Value{
14                     config::override::Priority::High,
15                     L"foobar"_s } );
16 // Устанавливаем переопределения в конфигурацию приложения.
17 config::override::OverrideConfiguration( std::move( settings ) );
```

Следующий вызов `OverrideConfiguration` с другим составом переопределений отменит предыдущие активные переопределения и применит новые.

- C++

```
1 // Формируем новый словарь переопределений.
2 config::override::Settings settings;
3 settings.emplace( L"Section.AnotherParameter"_s,
4                   config::override::Value{
5                     config::override::Priority::High,
6                     L"foobar"_s } );
7 settings.emplace( L"Section.OtherParameter"_s,
8                   config::override::Value{
9                     config::override::Priority::High,
10                    L"[42, 137]"_s } );
11 // Устанавливаем новые переопределения в конфигурацию приложения.
12 config::override::OverrideConfiguration( std::move( settings ) );
13 // Переопределение значения параметра Section.Parameter из
   предыдущего примера теряет
14 // силу, а новое переопределение параметра Section.OtherParameter
   применяется к
15 // конфигурации.
```

Для возвращения конфигурации к состоянию до установки переопределений достаточно вызвать `OverrideConfiguration` с пустым словарём.

- C++

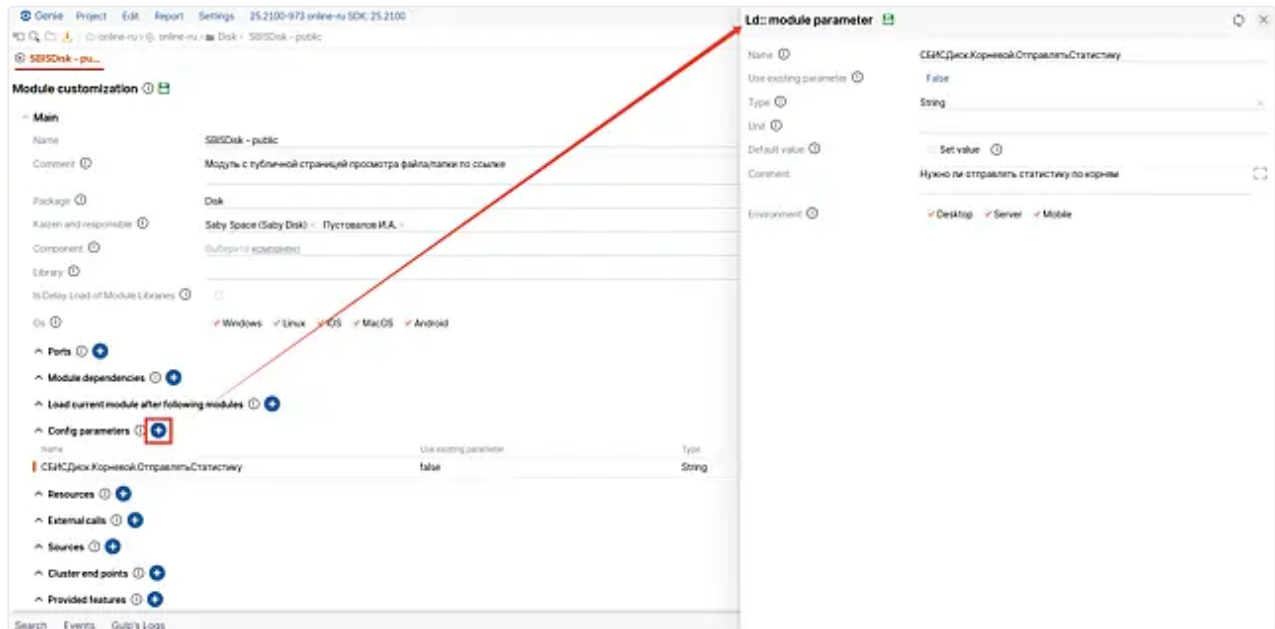
```

1 // Снимаем все переопределения.
2 config::override::OverrideConfiguration( config::override::Settings{}
);

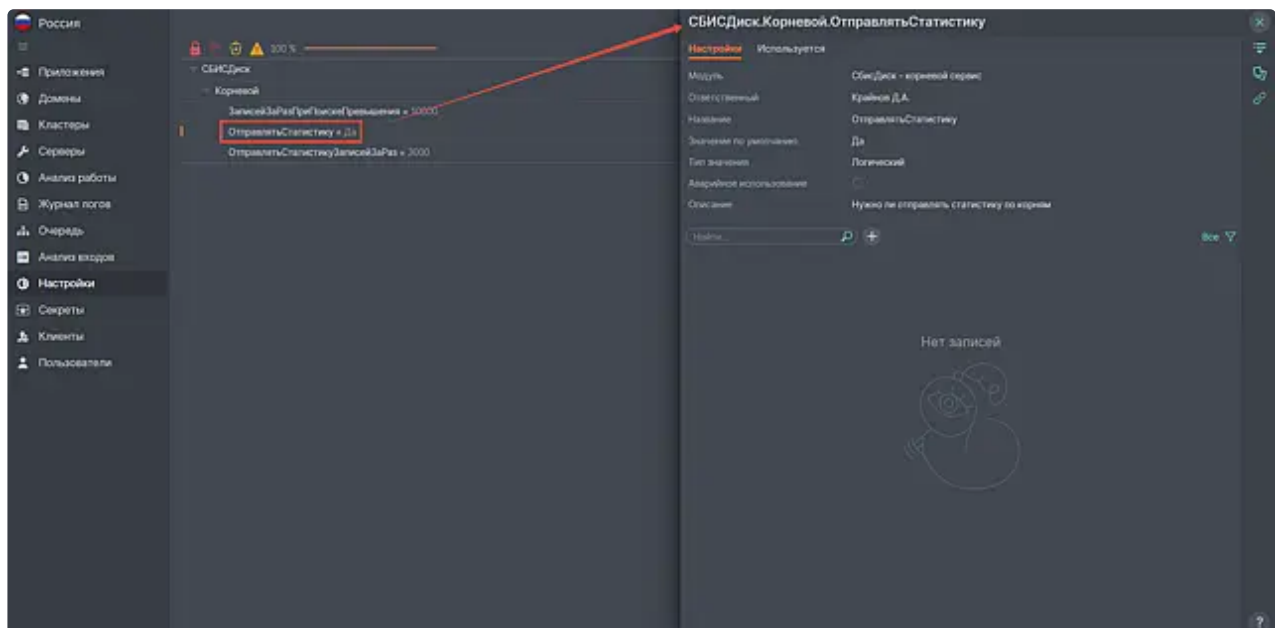
```

Пример 1. Получить значение параметра "СБИСДиск.Корневой.ОтправлятьСтатистику"

1. Создайте параметр в Genie:



2. Настройте его значение в сервисе "Управление облаком":



3. Из прикладного кода обратитесь к его значению:

- C++

```

1 String bdConf = Config::Instance().Get(
  L"СБИСДиск.Корневой.ОтправлятьСтатистику" );

```

- Python

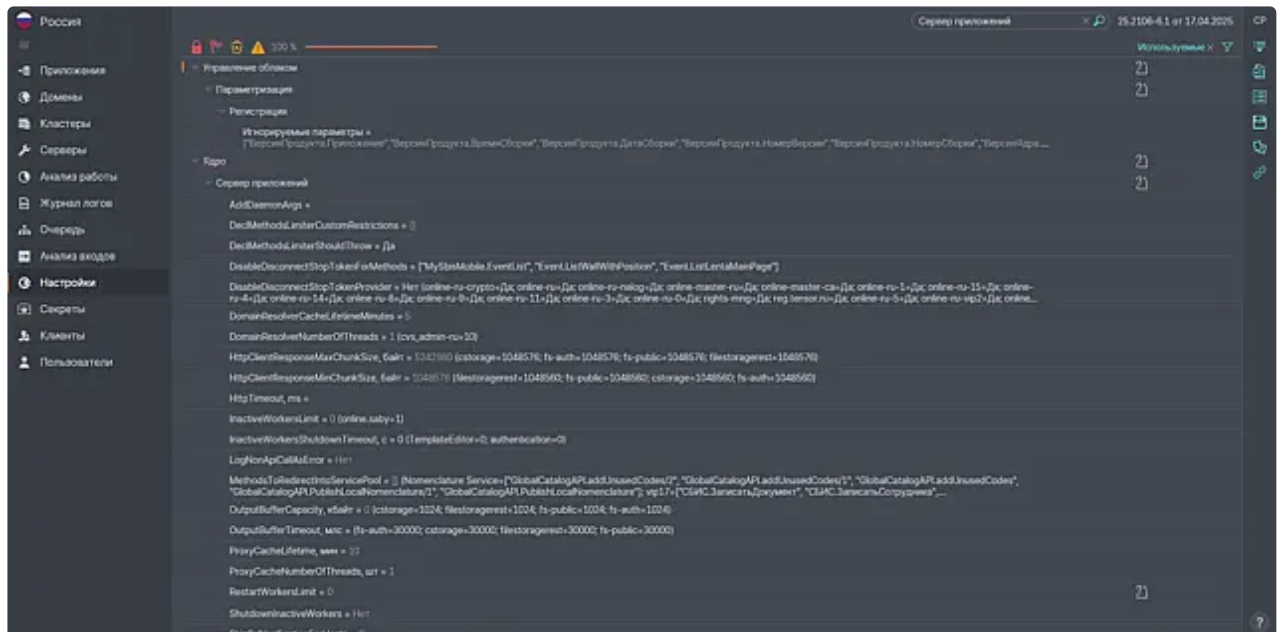
```

1 bdConf = Config.Instance().Get(
  "СБИСДиск.Корневой.ОтправлятьСтатистику" )

```

Пример 2. Обращение к несуществующему параметру

1. При обращении к несуществующему параметру (например, "Ядро.Сервер приложений.Очередь") будет возвращена пустая строка. На следующем скриншоте в списке отсутствует искомый параметр:



2. Допускается определение значения по умолчанию, которое будет возвращено вместо пустой строки при обращении к несуществующему параметру. Его задают вторым аргументом функции `Get`.

- C++

```
1 String bQueue = Config::Instance().Get( L"Ядро.Сервер приложений.Очередь", L"FIFO" );
```

- Python

```
1 bQueue = Config.Instance().Get( "Ядро.Сервер приложений.Очередь", "FIFO" )
```

Пример 3. Исключение при обращении к несуществующему параметру

1. Из [предыдущего примера](#) понятно, что при обращении к несуществующему параметру (например, "Ядро.Сервер приложений.Очередь") можно возвращать вместо пустой строки значение по умолчанию.
2. Также вместо значения по умолчанию можно генерировать исключение методом `GetOrThrow`.

- C++

```
1 String bQueue = Config::Instance().GetOrThrow( L"Ядро.Сервер приложений.Очередь" );
```

- Python

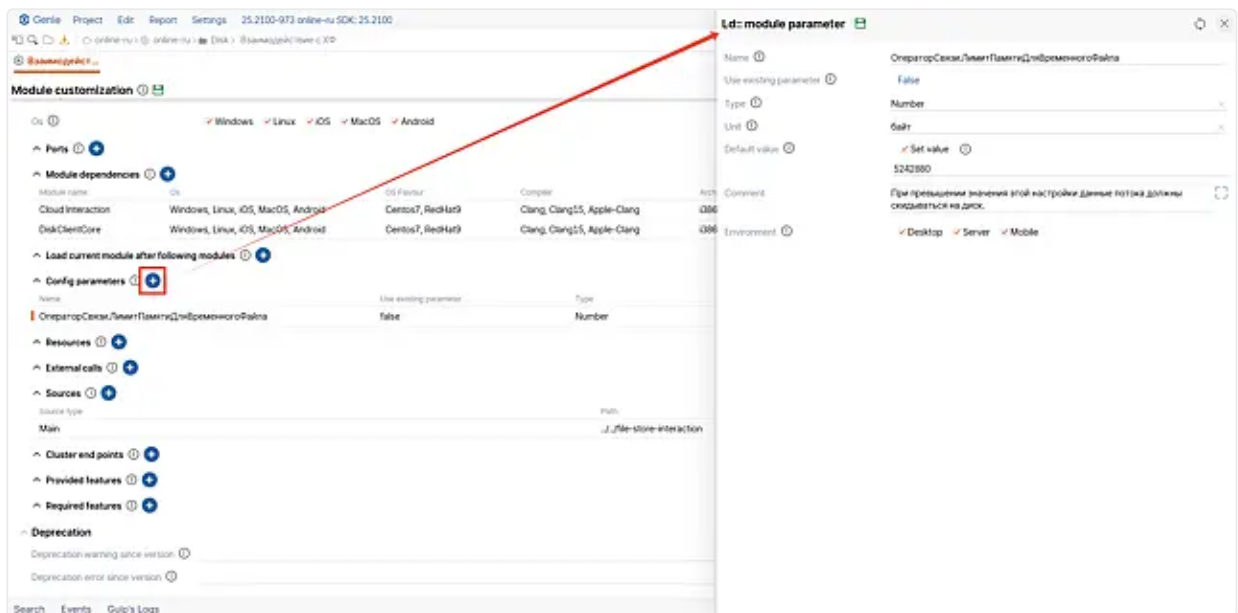
```
1 bQueue = Config.Instance().GetOrThrow( "Ядро.Сервер приложений.Очередь" )
```

Пример 4. Получение нестроковых значений параметров

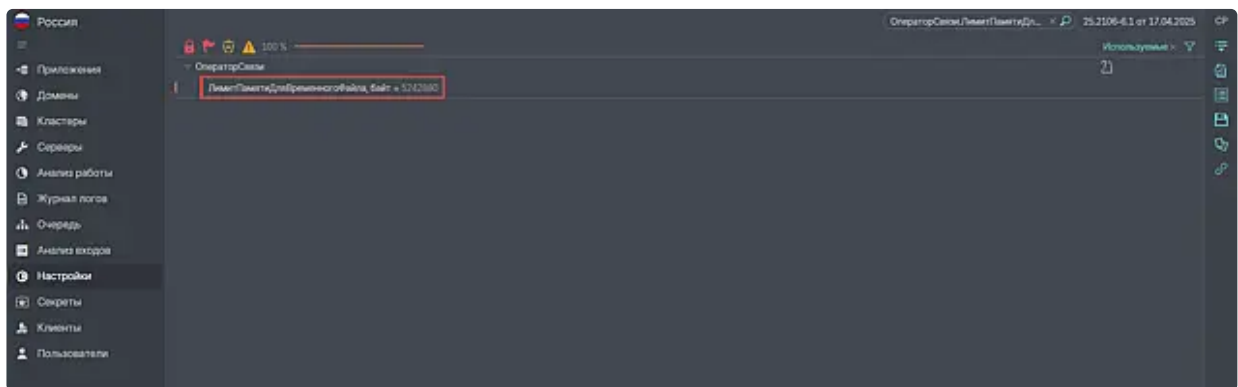
В этом примере рассмотрены случаи, когда значение параметра нужно получить не в виде строки.

- В случае C++ функция `Get` является шаблоном, что позволяет получать различные типы значений, а не только строку. Для примера сохраним значение в качестве целого числа.

1. Создайте параметр в Genie:



2. Настройте его значение в сервисе "Управление облаком":

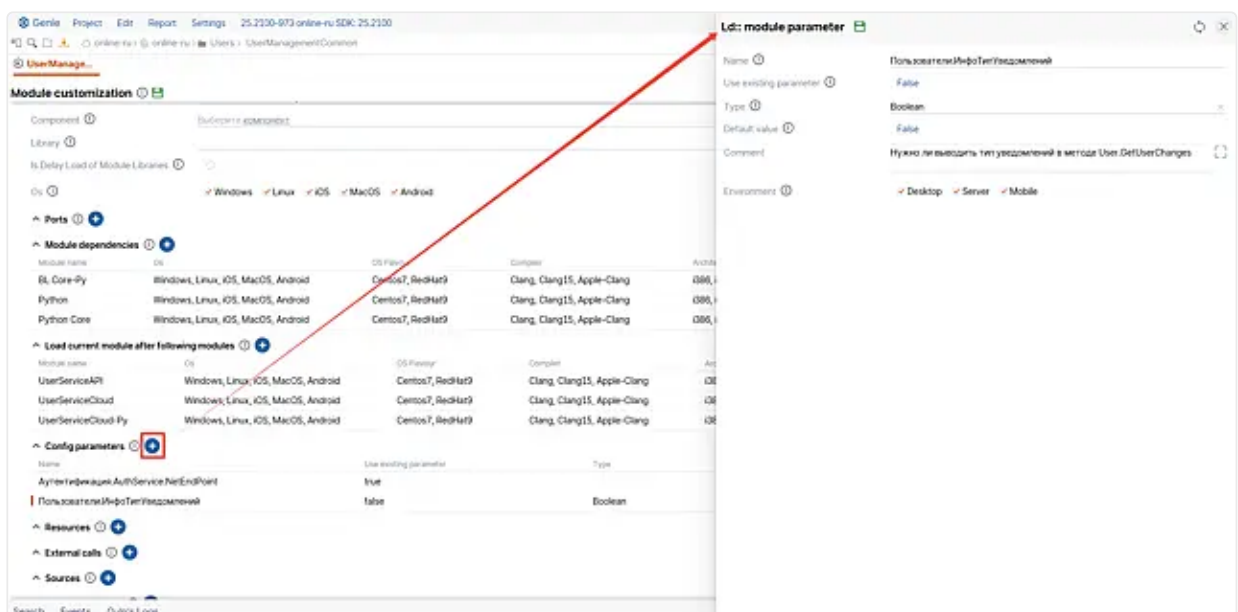


3. Обратитесь к значению из прикладного кода:

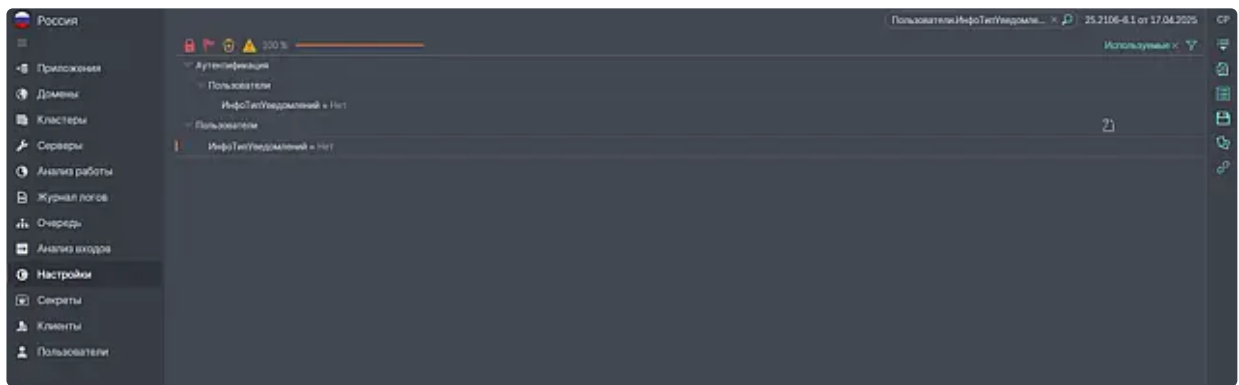
```
1 int const timeout = Config::Instance().Get<int>( L"Ядро.Асинхронные сообщения.AMQPTimeout");
```

- Для работы с логическими значениями в C++ есть отдельный метод — [GetBool](#).

1. Создайте параметр в Genie:



2. Настройте его значение в сервисе "Управление облаком":



3. Обратитесь к значению из прикладного кода:

```
1 bool serverLog = Config::Instance().GetBool(
  L"Ядро.Логирование.ЗаголовкиЗапросов" , false );
```

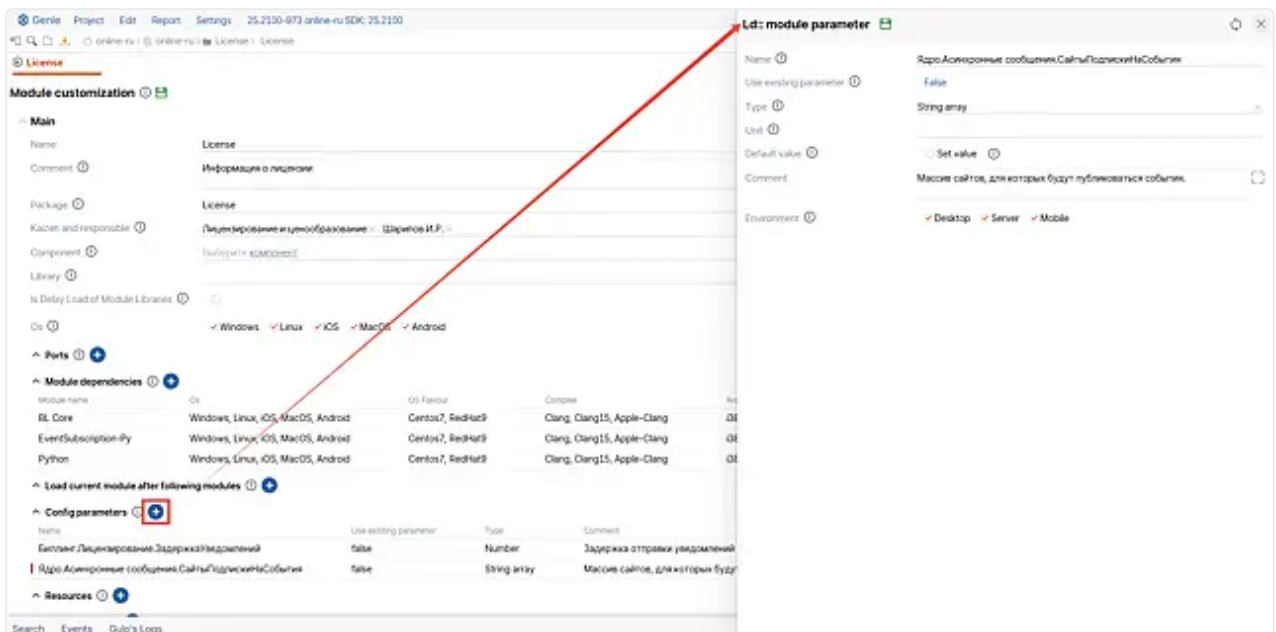
- В случае Python для каждого типа данных существует отдельный метод. Для примера, сохраните значение в качестве целого числа.

```
1 timeout = Config.Instance().GetInt32( "Ядро.Асинхронные
  сообщения.AMQPTimeout" )
```

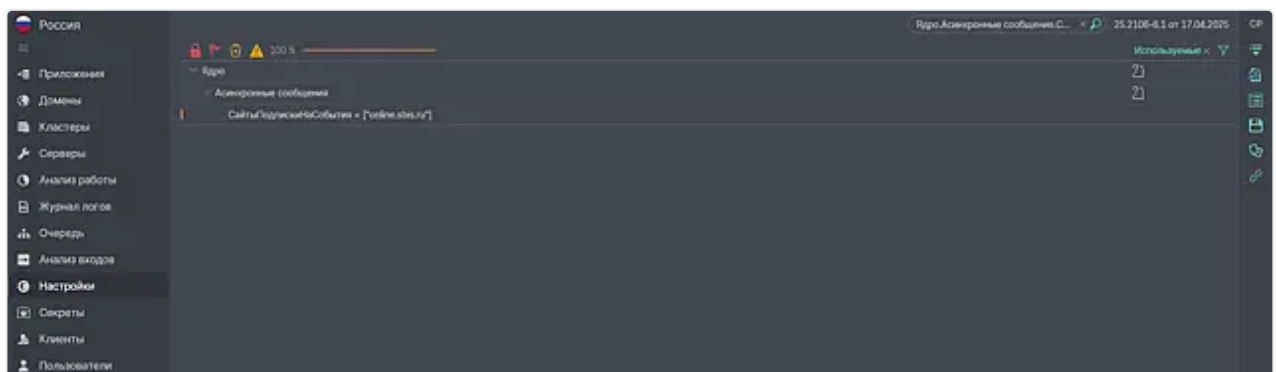
Полный список методов для Python можно найти в классе [Config](#).

Пример 5. В параметре передаётся список значений — массив строк

1. Создайте параметр в Genie:



2. Настройте его значение в сервисе "Управление облаком":



3. Обратитесь к его значению из прикладного кода:

- В случае C++ можно использовать тот же метод `Get`, чтобы получить значение, но при этом нужно явно указать получение массива строк — `ArrayString`.

```
1 ArrayString sites = Config::Instance().Get<ArrayString>(
  L"Ядро.Асинхронные сообщения.СайтыПодпискиНаСобытия", ArrayString()
);
```

- В случае с Python аналогичного C++ решения нет. Рекомендуется получение строки параметров конфигурации через метод [ConfigGet](#).

```
1 sites = ConfigGet( "Ядро.Асинхронные
  сообщения.СайтыПодпискиНаСобытия" )
```